

Full transcript

Java Exam Notes

Every prompt and every answer, in order

 Platform	ChatGPT
 Model	Unknown
 Total Messages	3 messages
 Exported On	29 September 2026 at 9:51
 Format	PDF (A4, Portrait)

USER

You said:

UNIT 2 Notes.pdf PDF

Java Unit 1_ Notes.pdf PDF

CDAP_Java_3rd_Sem_.pdf PDF

i am giving you the notes cdap the question paper and follow up i need you to give the best notes to prepare for exam

ASSISTANT

Absolutely. I checked the **question-paper pattern + Unit 1 notes + Unit 2 notes + CDAP/course document** you provided. I'll make these as **exam-preparation notes**, not just textbook notes.

Your paper is very predictable in structure:

Q	Marks	Unit	Topic	What you must prepare
Q1	2	I	Access & non-access specifiers	Definition + types/table
Q2	2	I	<code>final</code> keyword	3 uses
Q3	2	II	Abstract class & methods	Definition + syntax
Q4	2	II	Inner class	Definition + types
Q5	4	I	Constructors	Types + program
Q6	4	I	Inheritance	Types + program
Q7	4	II	Polymorphism	Overloading + overriding
Q8	4	II	Interface	Definition + implementation
Q9	8	I	Constructors + static + super + this + final + inheritance	Very important combined question
Q10	8	II	Packages	Very important

This also matches the official course structure: Unit I covers classes, objects, constructors, `this`, access/non-access specifiers, inheritance, `super`, abstract/final; Unit II covers polymorphism, interfaces and packages.

JAVA EXAM MASTER NOTES

PART A — UNIT I

1. ACCESS SPECIFIERS

Definition

Access specifiers control the visibility/accessibility of classes, variables, methods and constructors.

There are four main access levels:

- `private`
- default
- `protected`
- `public`

Your notes give the following access table:

Access Specifier	Same Class	Same Package	Subclass	Other Package
<code>private</code>	✓	✗	✗	✗
default	✓	✓	✗	✗
<code>protected</code>	✓	✓	✓*	✗*
<code>public</code>	✓	✓	✓	✓

Easy memory trick

Private → Class only

Default → Package

Protected → Package + Child

Public → Everywhere

`private`

Accessible only inside the class.

JAVA

```
class Student {  
    private int marks;  
  
    void setMarks(int m) {  
        marks = m;  
    }  
}
```

Used mainly for **data hiding and encapsulation** .

Default

If no access modifier is written:

JAVA

```
class Student {  
    int marks;  
}
```

It is accessible within the **same package** .

protected

Accessible:

- inside same class
 - same package
 - subclass
-

public

Accessible from anywhere.

JAVA

```
public int marks;
```

2. NON-ACCESS SPECIFIERS ★★ ★

Definition

Non-access specifiers provide additional behavior or characteristics to classes, methods and variables. They do **not primarily control visibility** .

Important ones in your Unit I:

- `static`
- `final`
- `abstract`

Your notes give a very useful memory trick:

① NOTE

static → **SHARED final** → **FIXED abstract** → **INCOMPLETE**

Exam answer

Q: What are non-access specifiers?

Non-access specifiers are keywords that define the behavior or characteristics of Java members rather than their accessibility. The important non-access specifiers are `static` , `final` and `abstract` .

3. `final` KEYWORD ★★ ★★ ★

This is directly Q2 for **2 marks** .

The `final` keyword is used to **restrict modification** . Its effect depends on whether it is applied to a variable, method or class.

Three uses

1. Final variable

Value cannot be changed.

JAVA

```
final int MAX = 100;
```

JAVA

```
MAX = 200;    // ERROR
```

2. Final method

Cannot be overridden.

JAVA

```
class Parent {
    final void display() {
        System.out.println("Hello");
    }
}
```

Child cannot redefine `display()` .

3. Final class

Cannot be inherited.

JAVA

```
final class Bank {
}
```

JAVA

```
class SBI extends Bank {
} // ERROR
```

★ Remember

<code>final</code> applied to	Result
Variable	Cannot reassign
Method	Cannot override
Class	Cannot inherit

This exact three-way distinction is emphasized in your notes.

Perfect 2-mark answer

① NOTE

The `final` keyword is used to restrict modification in Java. A final variable cannot be reassigned, a final method cannot be overridden, and a final class cannot be inherited.

4. CONSTRUCTORS ★ ★ ★ ★ ★

Q5 = 4 marks

A constructor is a special member of a class that is automatically called when an object is created. It is mainly used to initialize an object.

Characteristics

- Constructor name = class name.
- It has **no return type**.
- It is automatically called when an object is created.
- Constructors can be overloaded.
- Constructors are not inherited.

Types

A. No-argument constructor

A constructor that does not receive parameters.

JAVA

```
class Student {  
  
    Student() {  
        System.out.println("Student object created");  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        Student s = new Student();  
    }  
}
```

Output

OUTPUT

Student object created

When:

JAVA

```
new Student();
```

executes, `Student()` is automatically called.

B. Parameterized constructor

A constructor that accepts parameters to initialize an object.

JAVA

```
class Student {  
  
    int rollNo;  
    String name;  
  
    Student(int rollNo, String name) {  
        this.rollNo = rollNo;  
        this.name = name;  
    }  
  
    void display() {  
        System.out.println(rollNo);  
        System.out.println(name);  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
  
        Student s = new Student(101, "Rahul");  
  
        s.display();  
    }  
}
```

Output

OUTPUT

```
101  
Rahul
```

Constructor overloading

Having multiple constructors with different parameter lists.

JAVA

```
class Student {  
  
    Student() {  
        System.out.println("No arguments");  
    }  
  
    Student(int id) {  
        System.out.println("ID = " + id);  
    }  
  
    Student(int id, String name) {  
        System.out.println(id + " " + name);  
    }  
}
```

Key idea

TEXT

```
Same class  
  ↓  
Multiple constructors  
  ↓  
Different parameters  
  ↓  
Constructor overloading
```

5. **this** KEYWORD ★★★★★

Definition

this refers to the **current object**.

Your notes specifically explain its common use when constructor/method parameters have the same names as instance variables.

Example

JAVA

```
class Employee {  
  
    int id;  
    String name;  
  
    Employee(int id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
  
    void display() {  
        System.out.println(id);  
        System.out.println(name);  
    }  
}
```

Here:

JAVA

```
this.id
```

means the object's instance variable.

JAVA

```
id
```

means the constructor parameter.

Easy way

TEXT

```
this.id = id;  
  ↑    ↑  
object parameter
```

Uses of `this`

- Refers to current object.
- Distinguishes instance variable from parameter.
- Can call another constructor using `this()` .

- Can pass current object to a method/constructor.

6. **static** KEYWORD ★★★★★

Definition

static means the member belongs to the **class rather than individual objects** .

A static member is shared by all objects.

Static variable

JAVA

```
class Student {  
  
    static String college = "Alliance";  
  
    int rollNo;  
}
```

All Student objects share the same **college** .

Static method

JAVA

```
class Calculator {  
  
    static int square(int n) {  
        return n * n;  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
  
        System.out.println(Calculator.square(5));  
    }  
}
```

Output

OUTPUT

25

No object is required to call a static method.

Important

A static method **cannot directly access a non-static instance variable** .

7. INHERITANCE ★ ★ ★ ★ ★

Q6 = 4 marks

Definition

Inheritance is the mechanism by which a child class acquires accessible properties and methods from a parent class.

It promotes:

- code reuse
- hierarchical classification

Syntax

JAVA

```
class Child extends Parent {  
    // members  
}
```

Types of inheritance

Your syllabus specifically lists:

- Single
- Multilevel
- Hierarchical

Java does **not** support multiple inheritance through classes.

A. Single inheritance

TEXT

```
Animal  
↓  
Dog
```

JAVA

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
  
        Dog d = new Dog();  
  
        d.eat();  
        d.bark();  
    }  
}
```

Output

OUTPUT

```
Animal eats  
Dog barks
```

B. Multilevel inheritance

TEXT

```
Animal  
  ↓  
  Dog  
  ↓  
  Puppy
```

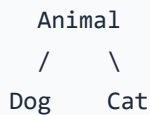
JAVA

```
class Animal {  
    void eat() {  
        System.out.println("Eating");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Barking");  
    }  
}  
  
class Puppy extends Dog {  
    void play() {  
        System.out.println("Playing");  
    }  
}
```

Puppy can access accessible members from Dog and Animal.

C. Hierarchical inheritance

TEXT



Multiple child classes inherit from one parent.

8. **super** KEYWORD ★★★★★

Definition

super refers to the **immediate parent class**.

It can be used to:

- Access parent variable.
- Call parent method.
- Call parent constructor.

1. Access parent variable

JAVA

```
class Person {  
  
    String name = "Person";  
}  
  
class Student extends Person {  
  
    String name = "Student";  
  
    void display() {  
        System.out.println(name);  
        System.out.println(super.name);  
    }  
}
```

OUTPUT

Student
Person

2. Call parent method

JAVA

```
class Animal {  
  
    void sound() {  
        System.out.println("Animal sound");  
    }  
}  
  
class Dog extends Animal {  
  
    void sound() {  
        super.sound();  
        System.out.println("Dog barks");  
    }  
}
```

3. Call parent constructor

JAVA

```
class Person {  
  
    Person() {  
        System.out.println("Person constructor");  
    }  
}  
  
class Student extends Person {  
  
    Student() {  
        super();  
        System.out.println("Student constructor");  
    }  
}
```

OUTPUT

```
Person constructor  
Student constructor
```

The notes specifically demonstrate that `super()` invokes the parent constructor first.

Q9 — MOST IMPORTANT 8-MARK QUESTION

Your question paper specifically says Q9 should cover:

NOTE

constructors, static, super, this, final and inheritance

So prepare this as one combined answer.

Model Question

Q. Explain constructors, this , static , super , final and inheritance in Java with suitable examples.

Answer structure

Write:

1. Constructor

Special member automatically called during object creation.

JAVA

```
Student(int id) {  
    this.id = id;  
}
```

2. **this**

Refers to current object.

JAVA

```
this.id = id;
```

3. **static**

Belongs to class and is shared.

JAVA

```
static String college = "Alliance";
```

4. **super**

Refers to immediate parent.

JAVA

```
super.display();
```

5. **final**

JAVA

```
final int MAX = 100;
```

- final variable → cannot change
- final method → cannot override
- final class → cannot inherit

6. Inheritance

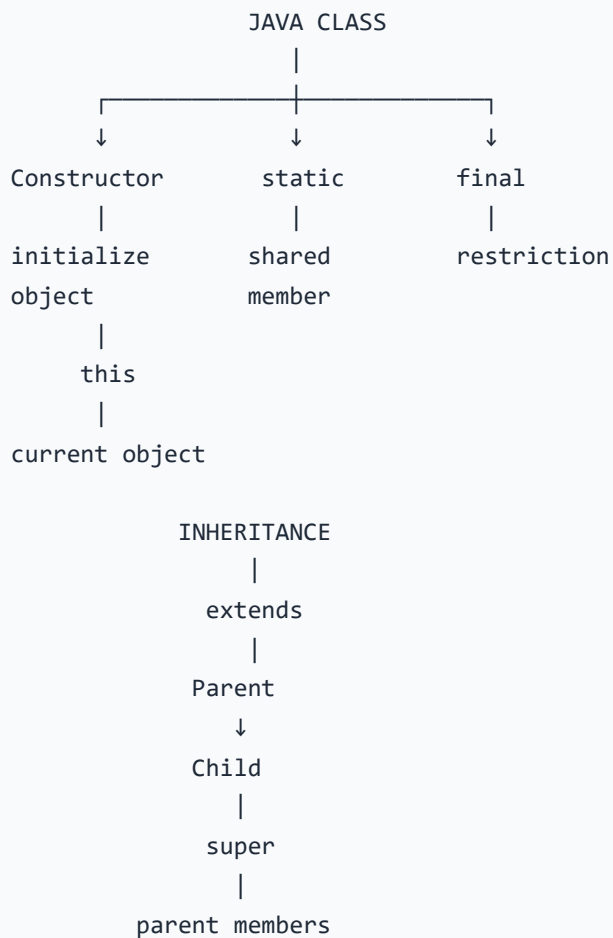
JAVA

```
class Dog extends Animal {  
}
```

Child gets accessible members from parent.

★ One diagram to remember

TEXT



The official Unit I exercise list itself includes programs involving parameterized constructors, static variables, inheritance, `super`, final methods/classes and abstract classes, so these are particularly worthwhile programs to practice.

PART B — UNIT II

9. POLYMORPHISM ★★★★★

Q7 = 4 marks

Definition

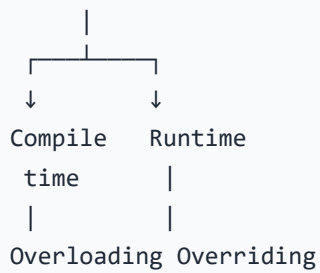
Polymorphism means **"many forms."**

In Java, the same method name or reference can behave differently depending on the situation.

There are two major types:

TEXT

Polymorphism



A. Method Overloading

Definition

Same method name but **different parameter lists** .

Parameters can differ by:

- number
- type
- order

Return type alone cannot overload a method.

Example

JAVA

```
class Calculator {  
  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
}
```

Output examples

TEXT

```
add(10,20)      → 30
add(10,20,30)   → 60
add(10.5,20.5) → 31.0
```

Remember

Overloading = same class + same name + different parameters

B. Method Overriding

Definition

When a subclass provides its own implementation of a method already defined in the superclass.

Method name and parameter list must be the same, with a compatible return type.

JAVA

```
class Animal {

    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {

    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
```

Runtime polymorphism

JAVA

```
Animal a;

a = new Dog();
a.sound();
```

OUTPUT

Dog barks

This is **dynamic method dispatch** .

★ Overloading vs Overriding

Feature	Overloading	Overriding
Time	Compile-time	Runtime
Parameters	Different	Same
Inheritance	Not required	Required
Class	Usually same	Parent + child
Method name	Same	Same
Binding	Static/early	Dynamic/late
Purpose	Different ways of performing operation	Change inherited behavior

This comparison directly follows your Unit II notes.

Memory trick

① NOTE

OverLOAD → different **LOAD** of arguments **OverRIDE** → child **RIDES** over parent's method

10. ABSTRACT CLASS & ABSTRACT METHOD ★★★★★

Q3 = 2 marks

Abstract class

A class declared using `abstract` .

JAVA

```
abstract class Animal {
}
```

It cannot normally be instantiated directly and may contain abstract and concrete methods.

Abstract method

A method declared **without a body** .

JAVA

```
abstract void sound();
```

A concrete child class must implement it.

Example

JAVA

```
abstract class Animal {  
  
    abstract void sound();  
  
    void eat() {  
        System.out.println("Animal eats");  
    }  
}  
  
class Dog extends Animal {  
  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

JAVA

```
Dog d = new Dog();  
  
d.sound();  
d.eat();
```

OUTPUT

```
Dog barks  
Animal eats
```

Important rules

- Cannot directly create object of abstract class.
- Abstract method has no body.

- Concrete child must implement abstract methods.
 - Abstract class can contain normal methods.
 - Abstract class can contain constructors.
 - If child does not implement all abstract methods, child must also be abstract.
-

11. INNER CLASS ★★☆☆

Q4 = 2 marks

Definition

An inner class is a class defined inside another class or interface.

The outer class is called the **enclosing class**.

Four types

- Member inner class
 - Static nested class
 - Local inner class
 - Anonymous inner class
-

Member inner class

JAVA

```
class Outer {  
  
    int number = 100;  
  
    class Inner {  
  
        void display() {  
            System.out.println(number);  
        }  
    }  
}
```

Creating object:

JAVA

```
Outer o = new Outer();
Outer.Inner i = o.new Inner();

i.display();
```

Static nested class

JAVA

```
class Outer {

    static class Inner {

        void display() {
            System.out.println("Static");
        }
    }
}
```

Object:

JAVA

```
Outer.Inner obj = new Outer.Inner();
```

No Outer object is required.

Local inner class

Declared inside a method.

JAVA

```
class Outer {  
  
    void display() {  
  
        class Local {  
            void show() {  
                System.out.println("Local");  
            }  
        }  
  
        Local obj = new Local();  
        obj.show();  
    }  
}
```

Anonymous inner class

Has **no class name** and is generally used when an object is required only once.

JAVA

```
abstract class Animal {  
    abstract void sound();  
}  
  
class Main {  
  
    public static void main(String[] args) {  
  
        Animal a = new Animal() {  
  
            void sound() {  
                System.out.println("Dog barks");  
            }  
        };  
  
        a.sound();  
    }  
}
```

12. INTERFACE ★★★★★

Q8 = 4 marks

Definition

An interface is a reference type used to define a **contract** that classes implement.

A class uses:

JAVA

implements

Syntax

JAVA

```
interface Vehicle {  
  
    void start();  
    void stop();  
}
```

Implementation:

JAVA

```
class Car implements Vehicle {  
  
    public void start() {  
        System.out.println("Car starts");  
    }  
  
    public void stop() {  
        System.out.println("Car stops");  
    }  
}
```

Main

JAVA

```
class Main {  
  
    public static void main(String[] args) {  
  
        Car c = new Car();  
  
        c.start();  
        c.stop();  
    }  
}
```

Output

OUTPUT

```
Car starts  
Car stops
```

Important interface rule ★

Interface abstract methods are implicitly `public` .

Therefore implementation must be:

JAVA

```
public void start()
```

Not:

JAVA

```
private void start()    // ERROR
```

Your notes explicitly emphasize this rule.

Multiple interfaces

A class can implement more than one interface.

JAVA

```
interface A {  
    void showA();  
}  
  
interface B {  
    void showB();  
}  
  
class C implements A, B {  
  
    public void showA() {  
        System.out.println("A");  
    }  
  
    public void showB() {  
        System.out.println("B");  
    }  
}
```

This is how Java supports **multiple inheritance of type** through interfaces.

Nested interface

An interface declared inside another class/interface.

JAVA

```
class Computer {  
  
    interface Processor {  
        void process();  
    }  
}
```

Implementation:

JAVA

```
class CPU implements Computer.Processor {  
  
    public void process() {  
        System.out.println("CPU processing");  
    }  
}
```

Access:

JAVA

`Computer.Processor`

Interface variables ★

Variables in an interface are automatically:

TEXT

```
public
static
final
```

Therefore:

JAVA

```
interface Constants {
    int MAX = 100;
}
```

is equivalent to:

JAVA

```
public static final int MAX = 100;
```

13. PACKAGES ★★★★★★

Q10 = 8 marks

This is one of the most important topics.

Definition

A package is a collection/namespace used to organize related Java classes and interfaces and avoid naming conflicts.

Examples:

TEXT

```
java.lang  
java.util  
java.io  
java.net  
java.sql  
java.time
```

Types of packages

1. Built-in/System packages

Provided by Java.

Examples:

TEXT

```
java.lang  
java.util  
java.io  
java.sql  
java.net  
java.time
```

2. User-defined packages

Created by the programmer.

Example:

TEXT

```
mypackage
```

Creating a package

Step 1 — Create package class

```
Student.java
```

JAVA

```
package college;

public class Student {

    public void display() {
        System.out.println("Student belongs to College");
    }
}
```

Step 2 — Compile

BASH

```
javac -d . Student.java
```

The `-d` option creates the `.class` file according to package structure.

It creates:

TEXT

```
college/
  Student.class
```

Accessing a package

Method 1 — Fully qualified name

JAVA

```
college.Student s = new college.Student();
```

Method 2 — `import`

JAVA

```
import college.Student;
```

Then:

JAVA

```
Student s = new Student();
```

Syntax:

JAVA

```
import packageName.ClassName;
```

or

JAVA

```
import packageName.*;
```

Built-in packages — MUST MEMORIZE

Package	Purpose	Examples
<code>java.lang</code>	Basic Java classes	String, Math, Object
<code>java.util</code>	Utilities/collections	ArrayList, HashMap, Scanner
<code>java.io</code>	Input/output	File, FileReader
<code>java.net</code>	Networking	URL, Socket
<code>java.sql</code>	Database	Connection, ResultSet
<code>java.time</code>	Date/time	LocalDate, LocalTime
<code>java.awt</code>	GUI	Frame, Button
<code>javax.swing</code>	GUI	JFrame, JButton

Special point

`java.lang` is automatically imported.

So:

JAVA

```
String name = "Java";
```

works without:

JAVA

```
import java.lang.String;
```

User-defined package complete program ★ ★ ★ ★ ★

File 1: `Calculator.java`

JAVA

```
package mypackage;

public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }

    public int multiply(int a, int b) {
        return a * b;
    }
}
```

File 2: `Main.java`

JAVA

```
import mypackage.Calculator;

public class Main {

    public static void main(String[] args) {

        Calculator c = new Calculator();

        System.out.println("Addition = " + c.add(10, 20));
        System.out.println("Multiplication = " + c.multiply(10, 20));
    }
}
```

Output

OUTPUT

```
Addition = 30
Multiplication = 200
```

This complete two-file package example is included in your Unit II notes.

Q10 — 8-MARK ANSWER FORMAT

Question

Explain packages in Java. Discuss built-in and user-defined packages and explain how to create, compile and access a user-defined package with an example.

Write in this order:

1. Definition — 1 mark

A package is a collection/namespace of related Java classes and interfaces used for organization and avoiding naming conflicts.

2. Advantages — 2 marks

- Avoids naming conflicts
- Provides access protection
- Better organization
- Code reusability
- Easier maintenance

These benefits are explicitly listed in the notes.

3. Types — 1 mark

- Built-in
- User-defined

4. Creating package — 1 mark

JAVA

```
package mypackage;
```

5. Compilation — 1 mark

BASH

```
javac -d . Calculator.java
```

6. Import/access — 1 mark

JAVA

```
import mypackage.Calculator;
```

7. Program/output — 1 mark

Give the Calculator example above.

This structure is enough to make a strong 8-mark answer.

LAST-MINUTE KEYWORD TABLE

Memorize this table before the exam:

Keyword	Meaning
<code>class</code>	Creates class
<code>new</code>	Creates object
<code>this</code>	Current object
<code>static</code>	Belongs to class
<code>final</code>	Restriction
<code>abstract</code>	Incomplete/abstraction
<code>extends</code>	Inherit/extend class
<code>super</code>	Immediate parent
<code>interface</code>	Defines contract
<code>implements</code>	Implements interface
<code>package</code>	Declares package
<code>import</code>	Imports package/class
<code>public</code>	Accessible everywhere
<code>private</code>	Class only
<code>protected</code>	Package + subclass

Your Unit II notes provide essentially this keyword mapping as the final revision section.



WHAT TO STUDY FIRST

Based on **your actual question paper** , don't spend equal time on everything.



Priority 1 — MUST MASTER

Q9 — 8 marks

Learn:

TEXT

```
Constructor
  ↓
this
  ↓
static
  ↓
inheritance
  ↓
super
  ↓
final
```

Q10 — 8 marks

TEXT

```
Package
  ↓
Types
  ↓
Advantages
  ↓
package keyword
  ↓
javac -d .
  ↓
import
  ↓
User-defined package program
```



Priority 2 — Very important

Q7 — Polymorphism

Know perfectly:

TEXT

Polymorphism

- └─ Overloading → Compile time
- └─ Overriding → Runtime

Q8 — Interface

Know:

TEXT

interface
implements
single interface
multiple interfaces
nested interface
interface variables

The CDAP specifically lists overloading/overriding, interfaces, nested interfaces, built-in packages and user-defined/nested packages as Unit II learning outcomes.

● Priority 3

Q5 — Constructors

Know:

- definition
- characteristics
- no-argument
- parameterized
- constructor overloading
- `this`

Q6 — Inheritance

Know:

TEXT

Single
Multilevel
Hierarchical

and `extends` .

Priority 4 — Easy 2 marks

Q1

Access specifiers:

TEXT

```
private
default
protected
public
```

Q2

Final:

TEXT

```
final variable → no reassignment
final method   → no overriding
final class    → no inheritance
```

Q3

Abstract:

TEXT

```
abstract class → cannot directly instantiate
abstract method → no body
```

Q4

Inner classes:

TEXT

```
Member
Static nested
Local
Anonymous
```

THE 10 QUESTIONS I WOULD PREPARE

If you want maximum exam efficiency, prepare these **exact questions** :

2 Marks

- Q1.** Explain access and non-access specifiers in Java.
- Q2.** Explain the `final` keyword with its three uses.
- Q3.** Define an abstract class and abstract method with syntax.
- Q4.** What is an inner class? List its types.

4 Marks

- Q5.** What is a constructor? Explain no-argument and parameterized constructors with a Java program.
- Q6.** Explain single, multilevel and hierarchical inheritance with suitable examples.
- Q7.** Explain compile-time and runtime polymorphism with overloading and overriding examples.
- Q8.** What is an interface? Explain how a class implements an interface with a suitable program.

8 Marks

- Q9.** Explain constructors, `this` , `static` , inheritance, `super` and `final` keywords with suitable examples.
- Q10.** Explain packages in Java. Discuss built-in and user-defined packages and demonstrate creation, compilation and importing of a user-defined package.

★ One very important observation

Your **CDAP document confirms that Unit I and Unit II are not just theory units** . The practical outcomes explicitly require programs for constructors, inheritance, abstract/final classes, overloading/overriding, interfaces and packages.

So for your preparation, use this rule:

① NOTE

Every theory topic → learn Definition + Syntax + Working + 1 small program + Output.

That is the safest way to prepare for both **written exam + coding/practical questions** .

If you study the notes above in this order, **Q1–Q10 are covered according to the question paper you gave** , rather than wasting time on topics outside the displayed pattern.

USER

You said:

think this as last minute revision i need prefect exam focused definition for
:class,object,super,final,static,polymorphism,abstraction,interface,packages